Student Platform — MERN Stack Folder & File Structure

Stack: React (client) + Node.js/Express (server) + MongoDB (database)
Core features covered:
Internships & Hackathons (Unstop/Devfolio-style)
Interview Prep (subject-wise playlists + PYQs)
Progress Tracking

Root Structure

```
student-platform/
├── client/            # React frontend
├── server/            # Node/Express backend
├── .gitignore
├── README.md
└── package.json       # root-level, runs client+server together via "concurrently"
```

1. Client (React Frontend)

```
client/
├── public/
│   ├── index.html
│   ├── favicon.ico
│   └── manifest.json
│
├── src/
│   ├── assets/
│   │   ├── images/
│   │   ├── icons/
│   │   └── styles/
│   │       ├── globals.css
│   │       └── variables.css
│   │
│   ├── components/              # Reusable UI, grouped by feature
│   │   ├── common/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Footer.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   ├── Loader.jsx
│   │   │   ├── Button.jsx
│   │   │   ├── Card.jsx
│   │   │   ├── Modal.jsx
│   │   │   ├── SearchBar.jsx
│   │   │   ├── FilterPanel.jsx
│   │   │   ├── Pagination.jsx
│   │   │   └── ProtectedRoute.jsx     # blocks routes if not logged in
│   │   │
│   │   ├── opportunities/          # Internships + Hackathons
│   │   │   ├── OpportunityCard.jsx
│   │   │   ├── OpportunityDetails.jsx
```

```
│   │   ├── OpportunityFilters.jsx  # filter by type/location/stipend
│   │   ├── ApplyButton.jsx
│   │   ├── BookmarkButton.jsx
│   │   └── CountdownTimer.jsx     # deadline countdown
│   │
│   ├── interviewPrep/
│   │   ├── SubjectCard.jsx       # DSA, OS, DBMS, CN, OOPs, Aptitude
│   │   ├── PlaylistCard.jsx
│   │   ├── VideoPlayer.jsx
│   │   ├── PYQList.jsx
│   │   ├── PYQCard.jsx
│   │   ├── TopicAccordion.jsx
│   │   └── QuizComponent.jsx      # optional: quick self-test
│   │
│   ├── progress/
│   │   ├── ProgressBar.jsx
│   │   ├── ProgressChart.jsx      # e.g. recharts pie/bar
│   │   ├── StatsCard.jsx
│   │   ├── StreakTracker.jsx
│   │   ├── ActivityHeatmap.jsx    # GitHub-style contribution grid
│   │   └── AchievementBadge.jsx
│   │
│   └── auth/
│       ├── LoginForm.jsx
│       ├── SignupForm.jsx
│       └── ForgotPassword.jsx
│
├── pages/                  # Route-level screens
│   ├── Home.jsx
│   ├── Login.jsx
│   ├── Signup.jsx
```

```
│   │   ├── Profile.jsx
│   │   │
│   │   ├── opportunities/
│   │   │   ├── OpportunitiesHome.jsx   # combined feed
│   │   │   ├── InternshipsPage.jsx
│   │   │   ├── HackathonsPage.jsx
│   │   │   ├── OpportunityDetailPage.jsx
│   │   │   └── MyApplications.jsx
│   │   │
│   │   ├── interviewPrep/
│   │   │   ├── InterviewPrepHome.jsx
│   │   │   ├── SubjectPage.jsx       # lists playlists+PYQs for one subject
│   │   │   ├── PlaylistPage.jsx
│   │   │   ├── PYQPage.jsx
│   │   │   └── CompanyWisePrep.jsx    # optional: TCS/Infosys/Amazon etc.
│   │   │
│   │   ├── progress/
│   │   │   └── ProgressDashboard.jsx
│   │   │
│   │   └── admin/                # for posting/managing content
│   │       ├── AdminDashboard.jsx
│   │       ├── ManageOpportunities.jsx
│   │       ├── ManageContent.jsx      # subjects/playlists/PYQs
│   │       └── ManageUsers.jsx
│   │
│   ├── context/
│   │   ├── AuthContext.jsx
│   │   └── ThemeContext.jsx
│   │
│   ├── hooks/
│   │   ├── useAuth.js
```

```
│   │   ├── useFetch.js
│   │   └── useProgress.js
│   │
│   ├── services/              # axios API layer
│   │   ├── api.js             # axios instance + interceptors
│   │   ├── authService.js
│   │   ├── opportunityService.js
│   │   ├── interviewPrepService.js
│   │   └── progressService.js
│   │
│   ├── routes/
│   │   └── AppRoutes.jsx
│   │
│   ├── utils/
│   │   ├── constants.js
│   │   ├── helpers.js
│   │   └── validators.js
│   │
│   ├── App.jsx
│   ├── index.js
│   └── App.css
│
├── .env
└── package.json
```

2. Server (Node/Express Backend)



```
server/
├── config/
│   ├── db.js            # MongoDB connection (mongoose.connect)
│   └── cloudinary.js    # optional: for resumes/images
│
├── models/
│   ├── User.js
│   ├── Opportunity.js   # internship + hackathon, distinguished by "type" field
│   ├── Application.js   # tracks who applied to what
│   ├── Subject.js       # DSA, OS, DBMS, CN, OOPs, Aptitude...
│   ├── Playlist.js
│   ├── Video.js
│   ├── PYQ.js
│   ├── Progress.js
│   └── Bookmark.js
│
├── controllers/
│   ├── authController.js
│   ├── userController.js
│   ├── opportunityController.js
│   ├── applicationController.js
│   ├── subjectController.js
│   ├── playlistController.js
│   ├── pyqController.js
│   └── progressController.js
│
├── routes/
│   ├── authRoutes.js
│   ├── userRoutes.js
│   ├── opportunityRoutes.js
│   ├── applicationRoutes.js
```

```
│   ├── subjectRoutes.js
│   ├── playlistRoutes.js
│   ├── pyqRoutes.js
│   └── progressRoutes.js
│
├── middleware/
│   ├── authMiddleware.js    # verifies JWT
│   ├── adminMiddleware.js   # role-based access
│   ├── errorMiddleware.js
│   └── uploadMiddleware.js  # multer, for resume uploads
│
├── utils/
│   ├── generateToken.js
│   ├── sendEmail.js
│   └── validators.js
│
├── .env
├── server.js            # entry point
└── package.json
```

3. How Each Feature Maps to the Structure

Feature 1 — Internships & Hackathons

Model: `Opportunity.js` with a `type` field (`"internship" | "hackathon"`) instead of two separate collections — mirrors how Unstop handles mixed opportunity feeds, and keeps filtering/search logic unified.

Suggested fields: `title, type, organizer, description, eligibility, stipendOrPrize, mode (remote/onsite), location, deadline, applyLink, tags[], postedBy, status`

Application.js tracks `user, opportunity, appliedAt, status`

Routes: `GET /opportunities`, `GET /opportunities/:id`, `POST /opportunities` (admin), `POST /applications`, `GET /applications/me`

Feature 2 — Interview Prep

Subject.js → top-level categories (DSA, OS, DBMS, CN, OOPs, Aptitude, HR)

Playlist.js → curated video playlists per subject (`subject ref, title, source, videoIds[]`)

PYQ.js → previous year questions (`subject ref, company, year, question, solutionLink, difficulty, tags[]`)

Routes: `GET /subjects`, `GET /subjects/:id/playlists`, `GET /subjects/:id/pyqs`

Feature 3 — Progress Tracking

Progress.js → one doc per user per subject: `user ref, subject ref, completedVideos[], completedPYQs[], percentage, lastAccessed, streakCount`

Updated whenever a user marks a video/PYQ as done (from `PlaylistPage.jsx` / `PYQPage.jsx`)

`ProgressDashboard.jsx` aggregates all subjects into one view (charts, streaks, badges)

Routes: `GET /progress/me`, `PATCH /progress/:subjectId`

4. Suggested Core Packages

Client: `react-router-dom`, `axios`, `react-icons`, `recharts` (progress charts), `react-hook-form` + `yup` (forms/validation), `tailwindcss` or `styled-components`

Server: `express`, `mongoose`, `jsonwebtoken`, `bcryptjs`, `dotenv`, `cors`, `multer`, `express-validator`, `nodemon` (dev)

5. Root `package.json` (for running both together)



json

```
{
 "name": "student-platform",
 "scripts": {
  "client": "npm start --prefix client",
  "server": "npm run dev --prefix server",
  "dev": "concurrently \"npm run server\" \"npm run client\""
 },
 "devDependencies": {
  "concurrently": "^8.2.2"
 }
}
```

Notes

Keeping Internships + Hackathons in one `Opportunity` model (instead of two) means one feed, one filter UI, one set of admin screens — less duplication as you scale to more opportunity types (jobs, scholarships, coding contests) later.

The `admin/` pages are what you or your team would use to post new internships/hackathons and upload playlists/PYQs — without this, all content has to be added directly in MongoDB.

This structure scales cleanly if you later split into microservices, but for a single-team student project, this monolith-with-clear-folders approach is the right level of complexity.